AgileX Scout Mini Autonomous Navigation

with Wheel-Slip Detection and Correction

Review Explanation Guide

Simulation walkthrough, algorithms used, and why they were chosen

ROS 2 Jazzy · Gazebo Harmonic · Nav2 · Ground Station GUI

How to use this guide: read sections 1–3 for the story, section 8 for the live demo script, and section 9 for likely reviewer questions. Every number in this guide comes from the project's own files or recorded test runs.

1. THE PROJECT IN ONE MINUTE

What it is. A simulated AgileX Scout Mini (a four-wheel skid-steer robot) that drives itself around a civil-department floor. You click a place on a map, or press a station button, and the robot plans a path, avoids obstacles and gets there. It was built with ROS 2 Jazzy, the Gazebo Harmonic simulator and the Nav2 navigation stack.

What was added last. Real floors can be slippery. We made three patches of the floor slippery in the simulation, showed that the robot then loses track of where it is and fails the patrol, and wrote a *slip controller* that detects the slip, stops the wheels spinning, and corrects the position estimate.

Headline results. On the slippery floor, the full 23-goal patrol was completed in 0 of 4 runs with correction OFF and in 3 of 7 runs with correction ON (three more ON runs reached 21 of 23, one stopped at 5) on the earlier patch layout. After the corridor patches were moved, one run with correction ON completed 23 of 23 and one run with correction OFF completed 3 of 23 (a very small sample). On the normal floor the patrol is 23 of 23. Results are from simulation only and are not perfectly repeatable.

2. PROJECT AT A GLANCE

Item	Detail
Robot	AgileX Scout Mini 4WD skid-steer; track 0.612 m, wheel radius 0.08 m; CAD from SolidWorks exported to URDF
Sensors	2D LiDAR (360°, 20 Hz), IMU (100 Hz), front camera (640×480, 30 Hz)
Software	Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic, Nav2, robot_localization, SLAM Toolbox, PyQt5/OpenCV
Package	*assem2_robot*: launch files, URDF, worlds, maps, Nav2 and EKF configuration, mission scripts, Ground Station, slip controller
World	Civil-department floor: six rooms and a central corridor; seven stations S1–S7; slip variant with three $\mu = 0.04$ patches
Main scripts	*civil_interactive_nav.py* (Ground Station), *civil_nav2_mission.py* (23-leg patrol), *slip_control.py* (slip detection and correction), *generate_civil_world.py* / *generate_slip_world.py* (world generators)
Run	*./run_civil_nav.sh* (normal floor), *./run_slip_demo.sh* (slip floor + patrol)

3. ARCHITECTURE: WHO TALKS TO WHOM

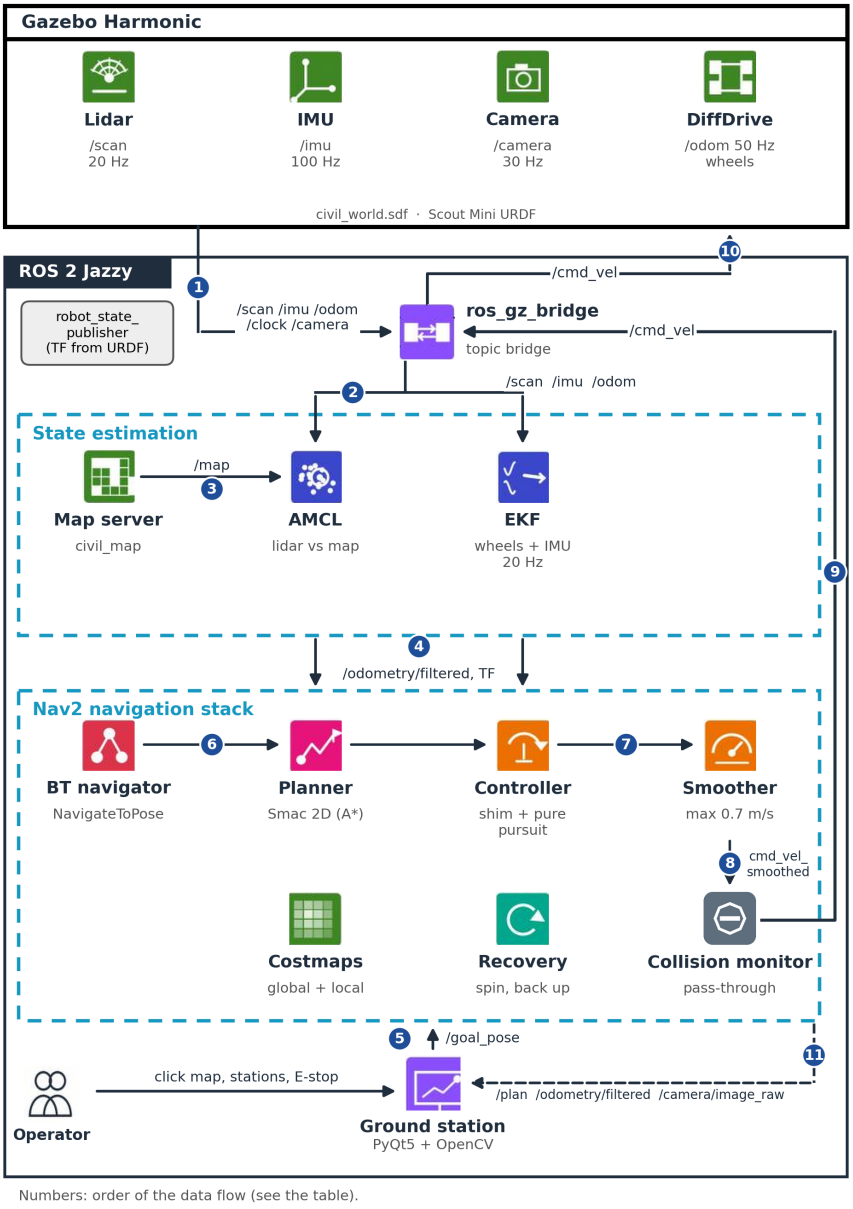


Figure 1. Complete architecture without slip control (numbers = order of the data flow; same as Fig. 14 of the report).

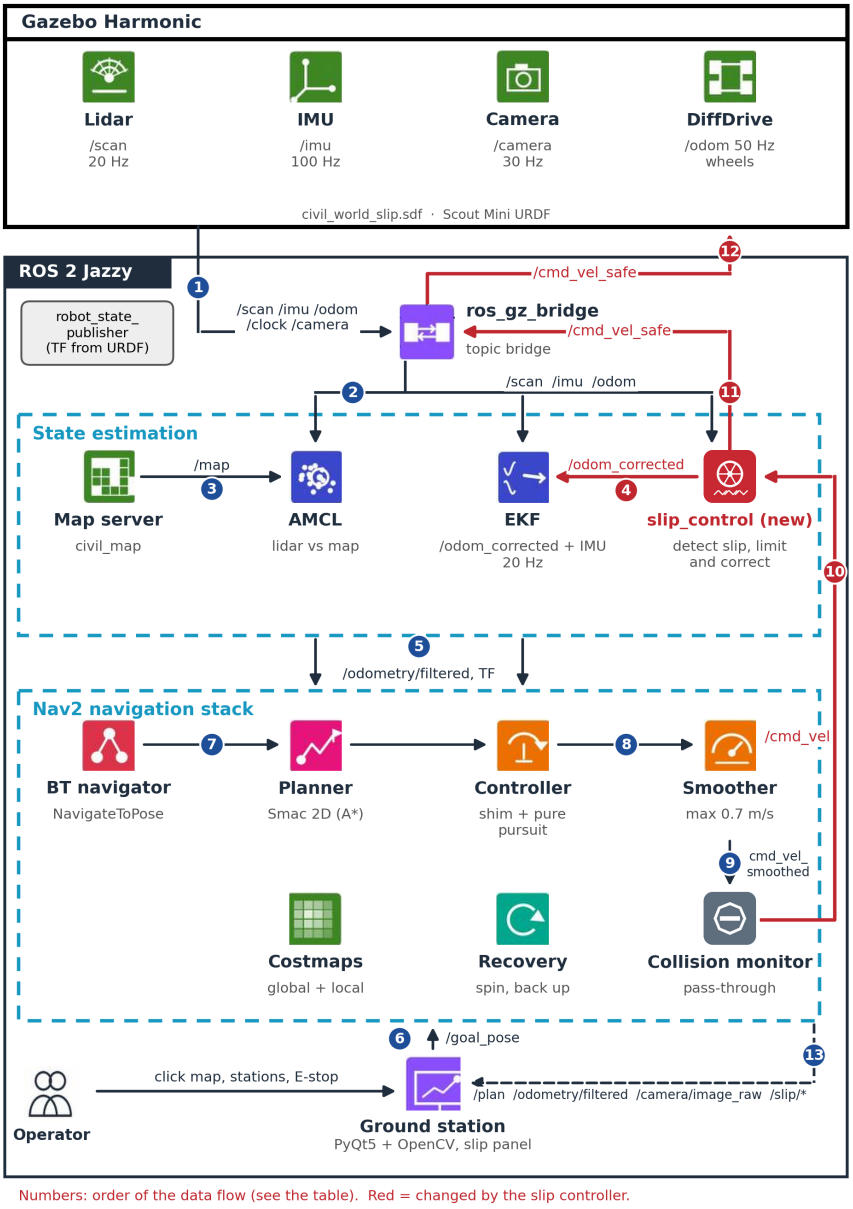


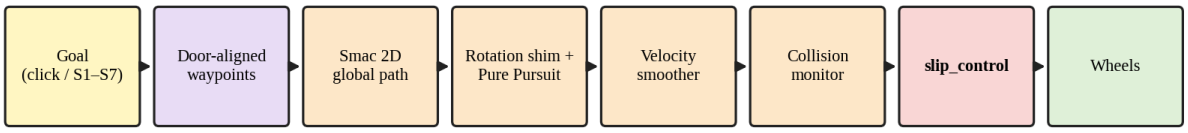
Figure 1b. Complete architecture with the slip controller. Red = changed by slip_control: commands pass through it and come out as /cmd_vel_safe; /odom_corrected feeds the EKF.

Read the diagram from left to right. **Gazebo** is the “real world”: it simulates physics, the floor, the robot and its sensors. **ros_gz_bridge** translates between Gazebo messages and ROS 2 topics. **State estimation** answers “where am I and how fast am I really moving?”. **Nav2** answers “how do I get there?”. The **Ground Station** is the human interface. Each box is a separate program (node), so any one of them can be replaced or tested alone — this is the main reason ROS 2 is used.

4. WHY EACH TECHNOLOGY WAS USED

Technology	Why we need it	Alternative and why not
ROS 2 Jazzy	Standard robot middleware: nodes, topics, actions, TF, launch files; large ecosystem (Nav2, SLAM Toolbox)	Writing our own messaging: slow and not reusable
Gazebo Harmonic	Simulates physics, sensors and friction, so ideas can be tested safely before hardware; friction patches make slip possible	Real robot: costly and risky for slip tests
URDF from SolidWorks CAD	Describes links, joints, sensors and the drive plugin so ROS and Gazebo share one robot model	Hand-built model: would not match the real Scout Mini
ros_gz_bridge	Connects Gazebo topics to ROS topics	Direct plugin: ties the code to one simulator
SLAM Toolbox	Builds the occupancy map once, by driving the robot around	Hand-drawn map: less accurate
EKF (robot_localization)	Combines wheel odometry and IMU for a smooth, drift-limited pose	Wheel odometry alone: drifts, especially when turning or slipping
AMCL	Uses the LiDAR scan and the saved map to remove drift	No map correction: error grows without bound
Nav2	Complete, tested stack for planning, following, recovery and safety	Writing a planner and controller from scratch
PyQt5 + OpenCV Ground Station	Click-to-navigate, live camera, slip display; makes demos and reviews easy	RViz only: no custom slip panel or station buttons
slip_control (own code)	Nav2 assumes wheel odometry is true; on slippery floor it is not. This node restores that assumption	Ignoring slip: patrol fails (0 of 4 runs complete)

5. HOW THE SIMULATION WORKS, STEP BY STEP



Every goal passes through the same chain; the slip controller is the last block before the wheels.

Figure 2. What happens after a goal is given.

- **Launch.** The launch file starts Gazebo with the world, publishes the robot description, starts the bridge, the EKF, AMCL, the map server, Nav2, *slip_control*, RViz and the Ground Station.
- **Localise.** AMCL places the robot on the saved map; the EKF keeps the pose smooth between scans.
- **Goal.** The operator clicks a map position or a station button. The Ground Station turns it into door-aligned waypoints if the target is in another room.
- **Plan.** Smac 2D finds a collision-free path on the costmap, which is the map with obstacles inflated by 0.55 m.
- **Follow.** Rotation-shim + Regulated Pure Pursuit picks a look-ahead point and produces a speed and turn rate; the velocity smoother limits acceleration; the collision monitor is the last Nav2 stage but its safety polygon is disabled, so it only passes commands through.

- **Protect the wheels.** *slip_control* compares wheel speed with the IMU-based ground speed and limits the command if the wheels spin.
- **Move.** The bridge delivers the command to Gazebo's DiffDrive plugin; the wheels turn; new sensor data comes back and the loop repeats about 20 times per second.
- **Arrive.** Nav2 accepts the goal within 0.20 m and 0.35 rad. In the patrol, a script then checks the true position in Gazebo.

6. THE SLIP PROBLEM: WHAT, WHY, HOW

6.1 What is wheel slip?

When a wheel turns but the robot does not move forward by the same amount, the wheel is slipping. On wet or polished floor a wheel can spin freely. Wheel odometry counts wheel turns, so it then reports a distance the robot never travelled. Nav2 and the EKF trust that number, so the robot's belief about its position drifts away from reality, and it fails to reach goals or hits walls.

6.2 How slip was simulated

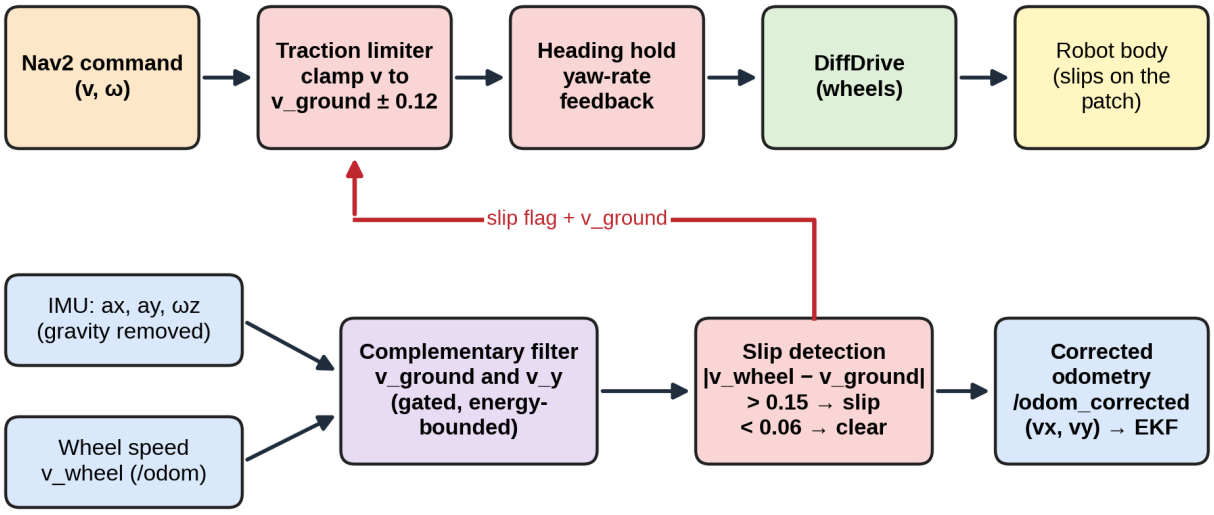
A low-friction material (μ = 0.04, against 0.25 for the wheels) was laid as three flush floor patches: the Conference Hall doorway and the east and west ends of the corridor. They are drawn blue in the 3D view and hatched blue on the Ground Station map. A raised patch was tried first, but the robot could not drive over its edge, so the patches were made flush with the floor.

6.3 How slip is detected

The IMU measures acceleration independently of the wheels. The node integrates it into a ground-speed estimate and blends it with wheel speed using a complementary filter, but only when the two agree about acceleration so that a spinning wheel cannot corrupt the estimate. Slip is flagged when |wheel speed − ground speed| exceeds 0.15 m/s for 0.05 s and cleared below 0.06 m/s. Two safeguards avoid false alarms: the estimate is bounded by an energy limit, and a latch breaker resets it if the body has been steady for 0.5 s while slip is still flagged.

6.4 How slip is corrected

Step	Action	Why
Traction limiter	While slipping and for 0.6 s after, clamp the wheel command to ground speed ± 0.12 m/s, never above the request	Like a car's traction control: less torque lets the tyres grip again
Heading hold	Feed back the IMU yaw rate (gain 0.4)	Uneven grip makes the robot veer; this steers it straight
Lateral estimate	Estimate sideways speed v_y from IMU acceleration minus the turning term	Wheel odometry cannot see sideways slides
Odometry correction	Publish /odom_corrected with the slip removed; the EKF uses it	The map pose then stays close to the true pose



Gazebo wheels follow velocity commands rigidly, so the limiter keeps the commanded acceleration within what the floor can deliver.

Figure 3. Inside the slip controller.

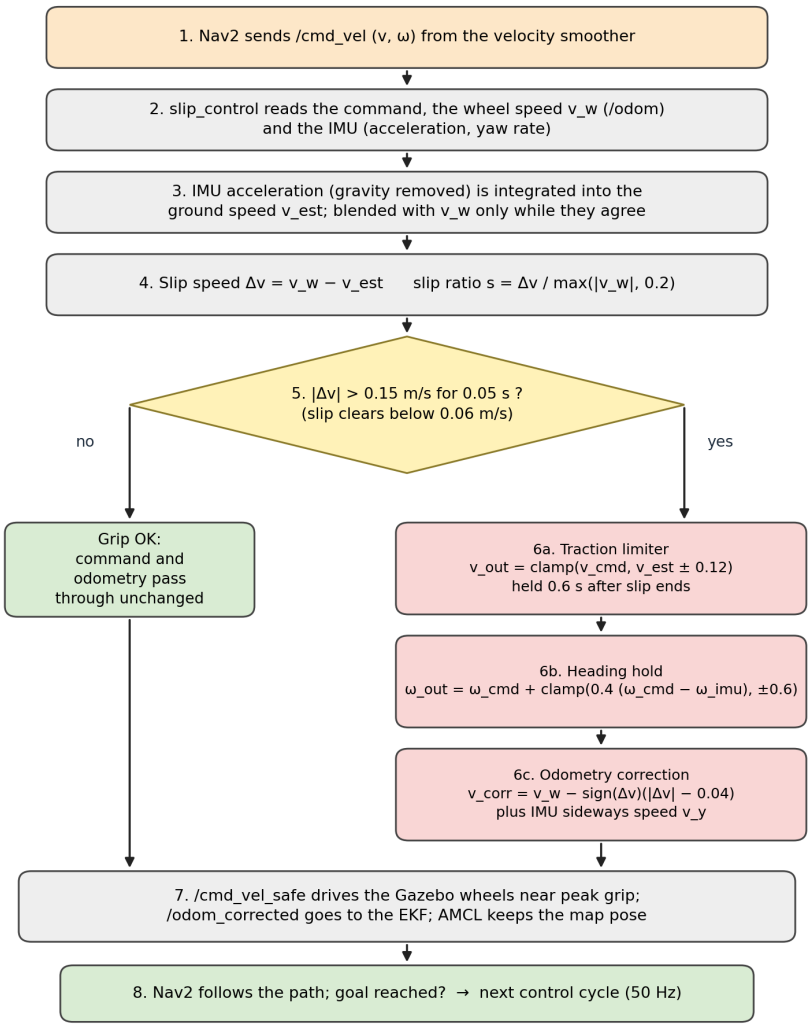


Figure 3b. Workflow of the control loop with slip correction (steps 1–8).

Key formulas. Slip speed $\Delta v = v_{wheel} - v_{ground}$. Slip is declared when $|\Delta v| > 0.15$ m/s for 0.05 s and cleared below 0.06 m/s. Slip ratio $s = \Delta v / \max(|v_{wheel}|, 0.2)$. Traction limit: $v_{out} = \text{clamp}(v_{cmd}, v_{ground} - 0.12, v_{ground} + 0.12)$. Heading hold: $\omega_{out} = \omega_{cmd} + \text{clamp}(0.4(\omega_{cmd} - \omega_{imu}), \pm 0.6)$. Corrected speed: $v_{corr} = v_{wheel} - \text{sign}(\Delta v)(|\Delta v| - 0.04)$. A patch with $\mu = 0.04$ gives at most $\mu \cdot g \approx 0.39$ m/s².

6.5 Correction ON versus OFF

The same slip world and the same 23-goal patrol were run with the correction switched on and off. With it OFF the node only monitors slip and passes commands and odometry through unchanged.

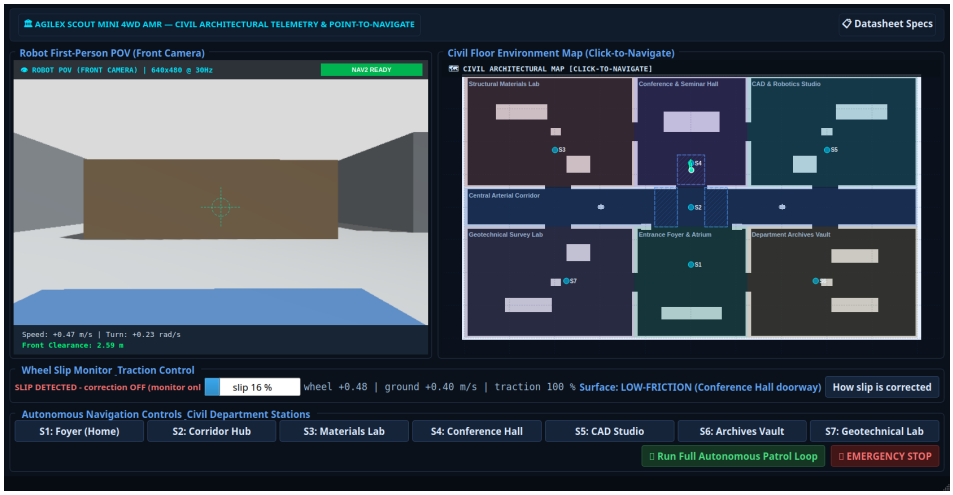


Figure 4. Correction OFF (final layout): wheels at 0.48 m/s, the robot at 0.40 m/s (slip 16 %), traction 100 % because nothing limits the wheels.

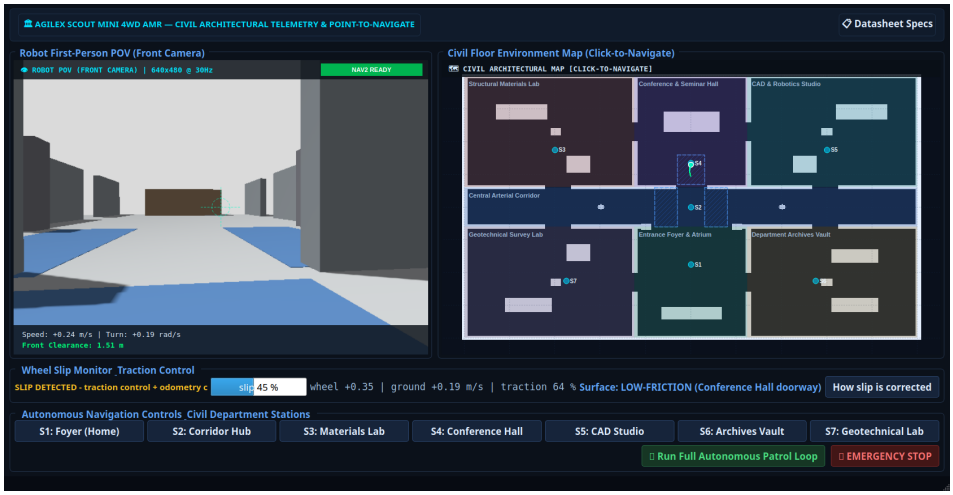


Figure 5. Correction ON: slip detected at 45 %; the wheel command is cut to 0.35 m/s against a ground speed of 0.19 m/s (traction 64 %).

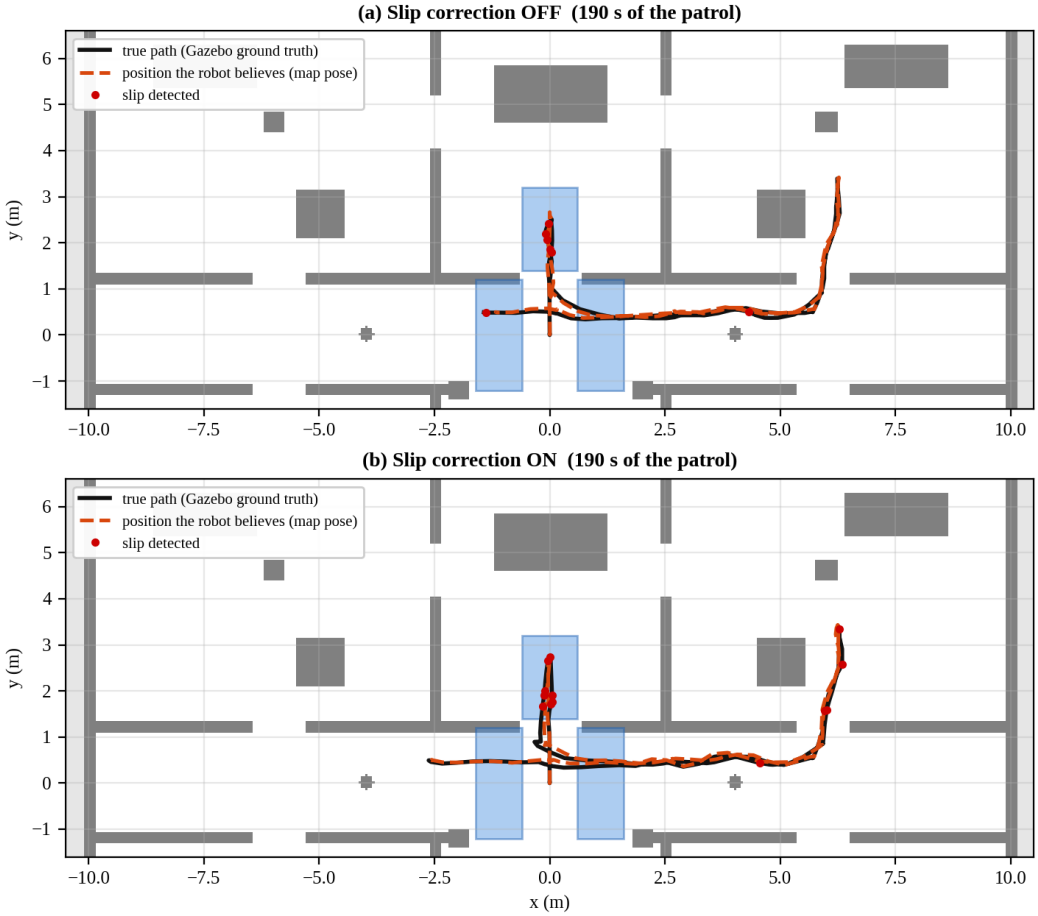


Figure 6. True path (black) against the position the robot believes (orange). Final layout, first 190 s, one run each. Both stay close to the true path; OFF is slightly worse.

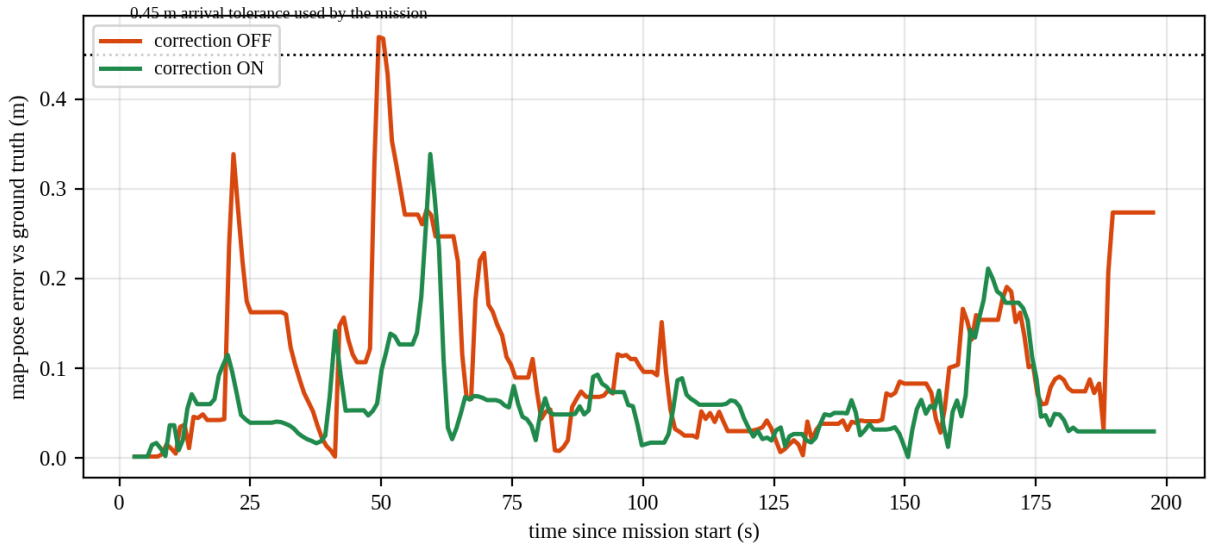


Figure 7. Localisation error against time. Final layout, 190 s: OFF peaks at 0.47 m, ON at 0.34 m (mean over first 100 s 0.13 m vs 0.07 m). The much larger errors (up to 3.9 m OFF) were on the earlier layout.

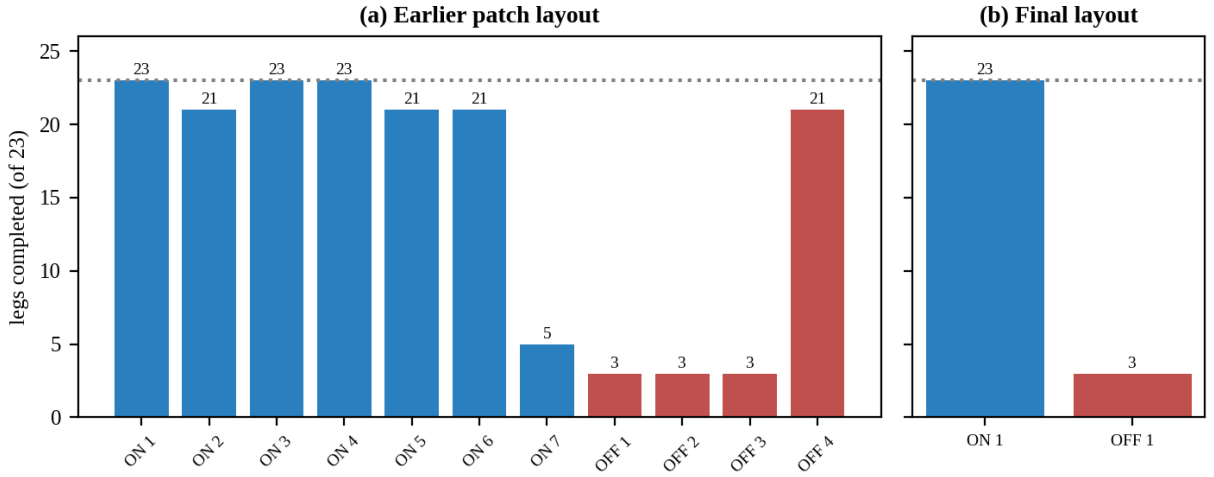


Figure 8. Legs completed per run (of 23). (a) earlier layout: ON 23, 21, 23, 23, 21, 21, 5; OFF 3, 3, 3, 21. (b) final layout: ON 23, OFF 3 (one run each). The spread is honest run-to-run variation.

Measure	OFF	ON
Full patrols completed	0 of 4	3 of 7
Typical legs completed	3 of 23	21–23 of 23
Max map-pose error (first 100 s), earlier layout	2.30 m	0.59 m
Mean map-pose error (first 100 s), earlier layout	0.60 m	0.07 m
Max / mean map-pose error (first 100 s), final layout	0.47 m / 0.13 m	0.34 m / 0.07 m
Straight-line test on μ = 0.02 floor	+1.12 m error	+0.04 m error

7. THE GROUND STATION

The Ground Station is the demonstration window. Top left: the robot's camera with speed, turn rate and front clearance. Top right: the map; click anywhere to navigate; the green line is the planned path; blue hatched boxes are slip zones. Middle: the **Wheel Slip Monitor** shows the slip percentage bar, wheel versus ground speed, traction scale, the surface under the robot, and a *How slip is corrected* button that

opens an explanation. If correction is off, the label says so in red. Bottom: stations S1–S7, patrol loop and emergency stop.

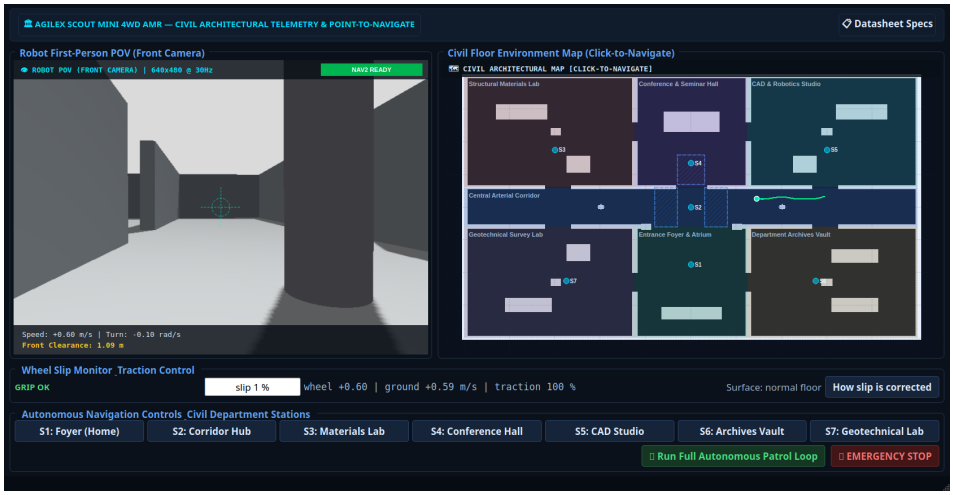


Figure 9. Ground Station with a planned path to the Conference Hall.

8. LIVE DEMO SCRIPT

Suggested order for a review of about ten minutes. Use a fresh start each time because the robot must begin at the world origin.

Step	Do	Say
1	*./run_civil_nav.sh* and wait for “Managed nodes are active” (30–60 s)	“This starts Gazebo, Nav2, localisation, RViz and the Ground Station on the normal floor.”
2	Click a station button, e.g. S5	“Nav2 plans through the doors; the green line is the plan; the camera shows what the robot sees.”
3	Click a point in another room on the map	“The Ground Station converts this to door-aligned waypoints so it works across rooms.”
4	Stop everything; start the slip floor with correction OFF: *./run_slip_demo.sh slip_correction:=false*	“Same world, but with three slippery patches (blue).”
5	Watch the slip panel when the robot enters the patch	“Wheels spin faster than the robot moves; the panel says slip and correction OFF; the robot stalls and loses position.”
6	Restart with correction ON: *./run_slip_demo.sh*	“Now the controller limits the wheel speed and corrects odometry; traction drops to about 50 % on the patch.”
7	Open *How slip is corrected*; show Figures 6–8	“On the earlier layout, three of seven full patrols with correction versus none of four without; on the final layout one run each (23 vs 3 legs). Not perfect, small sample, and I can explain the remaining failures.”

Safety net. If the live demo misbehaves, say so honestly and show the recorded figures. Run-to-run variation is real in this simulation (Section 10).

9. QUESTIONS A REVIEWER MAY ASK

Q. Why did you use ROS 2 and Nav2 instead of writing your own navigation?

Nav2 is the standard, tested stack for planning, following, recovery and safety, so effort could go into the new part, slip handling. ROS 2 also lets each part be a separate node and replaced independently.

Q. Why Gazebo?

It simulates physics, sensors and friction, so slip can be created on purpose and tested safely and repeatably before any hardware.

Q. How does the robot know where it is?

The EKF fuses wheel odometry and the IMU for a smooth pose, and AMCL matches the LiDAR scan to the saved map to remove drift.

Q. What is AMCL?

Adaptive Monte Carlo Localisation: a particle filter. Many guesses of the pose are scored against the LiDAR scan and the map; the best ones survive. Here 1000–4000 particles are used.

Q. Why Smac 2D and Pure Pursuit?

Smac 2D is a fast grid A* planner that handles narrow doors. Regulated Pure Pursuit follows a path smoothly and slows for curves and obstacles; a rotation shim first turns the robot toward the path when the heading error is over 0.52 rad.

Q. Why do the wheels slip in simulation?

Gazebo wheels follow velocity commands rigidly. On a patch with $\mu = 0.04$ the floor can only give a small acceleration (μg), so a command that accelerates faster makes the wheels slide.

Q. How do you detect slip when odometry itself is wrong?

By comparing it with an independent sensor: the IMU. The IMU acceleration is integrated to a ground-speed estimate. Slip is flagged when wheel speed and this estimate differ by more than 0.15 m/s.

Q. Why not just use the IMU for everything?

Integrated acceleration drifts. The complementary filter uses the IMU for short-term changes and the wheels for long-term accuracy, and only trusts the blend when the two agree.

Q. How does the correction help?

Three ways: it limits wheel speed so traction is regained; it feeds back yaw rate to keep heading; and it publishes slip-corrected odometry (including sideways speed) so the pose estimate stays close to the truth.

Q. Why not 100 % success with correction on?

Some remaining failures: the robot slides sideways into columns or walls, and localisation drifts. Results also vary between runs. Reported honestly: 3 of 7 full patrols, 3 reached 21 of 23, 1 stopped at 5.

Q. Did you test on a real robot?

No. Everything is in simulation. Thresholds would need re-tuning for real contact, but the method (IMU versus wheel speed) is the same one used in real traction control.

Q. What went wrong during development?

See Section 19 of the report: stale installed script, leftover processes corrupting TF, limiter runaway, false slip latch, the raised patch lip, sideways skid, click-to-navigate routing, and black screenshots on Wayland. Each has a fix recorded.

Q. How do you know the robot reached the goal?

The mission script compares the Gazebo ground-truth pose, not the robot's own estimate, against the goal within 0.45 m, and retries a leg up to three times.

Q. What would you do next?

Repeat the patrol many more times for statistics, test on hardware, add a controller that steers against sideways slides, and tune friction levels.

10. LIMITATIONS AND HONEST NOTES

- Simulation only; no hardware tests.
- Slip results vary from run to run (ON: 23, 21, 23, 23, 21, 21, 5 legs). The 5-leg run happened while a GUI capture was running.
- Remaining failures with correction ON: sideways slides and localisation drift.
- One OFF run reached 21 legs, so slip without correction is not always fatal.
- Gazebo and RViz windows could not be screen-captured on this Wayland desktop; Ground Station screenshots were rendered off-screen with Qt. Gazebo and RViz pictures in the report come from earlier sessions.
- During the OFF screenshot run the panel label changed colour partway through for an unknown reason, so only the early OFF frames are used.
- Nothing from the slip work is committed to git yet; *run_slip_demo.sh* wait logic is untested end to end.

11. GLOSSARY

Term	Meaning
Skid-steer	Turning by driving left and right wheels at different speeds; wheels scrub sideways during turns
Odometry	Position estimate from counting wheel turns
IMU	Inertial sensor: acceleration and rotation rate
EKF	Extended Kalman Filter: blends several noisy sensors into one estimate
AMCL	Particle-filter localisation against a known map
TF	ROS system of coordinate frames and their transforms
Costmap	Grid of how costly or forbidden each cell is for the robot
Inflation	Growing obstacles by a margin so the robot keeps clear
Complementary filter	Blend of a drifting-but-fast sensor and a slow-but-stable one
μ (mu)	Friction coefficient; the floor patches use 0.04
ZUPT	Zero-velocity update: resetting the speed estimate when the robot is certainly still
Slip ratio	Difference between wheel speed and true ground speed